

Project 1 - Documentation

By: Austin Emmanuel Paulraj

0. Design Plans

Included at the end as an appended set of images and handwritten descriptions.

Link to video showing full project - <https://youtu.be/-n9841k6pXo>

1. Project Description

The project depicts a scene of a small patch of grass. At the center is a tree, with bushes scattered around it. The idea for the scene comes from a time when I set up a small picnic in Minecraft, specifically in a biome with pretty trees, for a date. Keeping that theme, the aim for the implementation was to have an isometric or blocky style that makes it look like it is from an older pixel art game.

I felt this would be a good style to go for based on the fact that we are used to older, rasterized rendering techniques that don't account for aliasing of the line edges. That, combined with the smaller canvas sizes in level 2 or 3, gives the scene a very nostalgic feel. I was especially happy with the outcome of level 3, using triangles to fill in the full pixel colors.

The flat colors and heavily "crispy" edges reminded me of old pixel games, similar to the original Sonic the Hedgehog games, which take place on a very "green" plateau and forest-like environment (in the early levels). I also thought that the blocky design would allow for a stronger feeling of the scene being 3-Dimensional.

This strong "solid" feel makes up for some of the uncanny parallax visuals present in the usage of the pinhole camera method that I observed in the 1st assignment where we implemented a basic white wireframe block. Overall, I am quite happy with the outcome; as someone with a history of using software like Blender for 3D work, it was interesting to take a different approach in creating a 3D art piece.

2. Implementation & AI usage logs

The main implementation of the code was done by adapting code from the assignments as well as the article detailing triangle rasterization of pixel colors. In preparation for applying to AI Engineer jobs, the project was also used as a chance to show the usage of agentic AI in building a full project, from planning to deployment. As such, I will document each step of the implementation done and the AI usage with specifics on what it did and how I confirmed its correct working.

The first step was drawing out the plan as shown at the end of the document. In this plan, I included a key to show the exact measurements of each line in each shape. These values are not actually raw pixel lengths but rather relative size ratio coefficients. We use a variable titled “R” to act as a scaling factor on these coefficients.

This lets us keep the whole scene in scale, but also scale it up or down to easily fit the camera plane. Also, doing this dynamically makes it so that we don’t have to hard-calculate (or worse, re-calculate) the vertex positions of each box corner, just to change the scaling of the objects. I knew I wanted the scene building process to be dynamic, so I used Cursor AI to plan out a method in JavaScript to do this.

The use of AI made this part much easier, as I am a Python specialist (AI/ML specialization), so the specifics of syntax and handling basic, low-level errors were made many times more efficient. For the implementation of dynamic scene building, I prompted with the plan of making it very similar to how you would handle dynamic level building in Game Design. The required shapes are defined as individual world/position agnostic object.

These are then “spawned” as instances at the position and scale defined for the scene/level. I defined this logic to the agent, and it was able to generate the applicable code. I cross-referenced its working by loading example instances. The initial file included the base code for the “box wireframe” drawing project and the scene-building functions. The drawline and draw functions were what I used as a baseline to start adding on top of for further functionality.

Project 1: Block Tree | Austin Paulraj

Level 3: Triangles and triangle fill

Arrow keys to move camera, R to reset camera

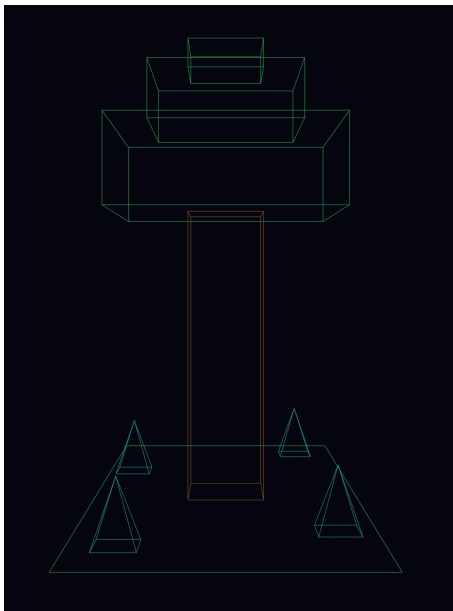
0 key - Level 0: Project preliminaries

1 key - Level 1: Wire-frame, pinhole camera

2 key - Level 2: Lower resolution, line drawing

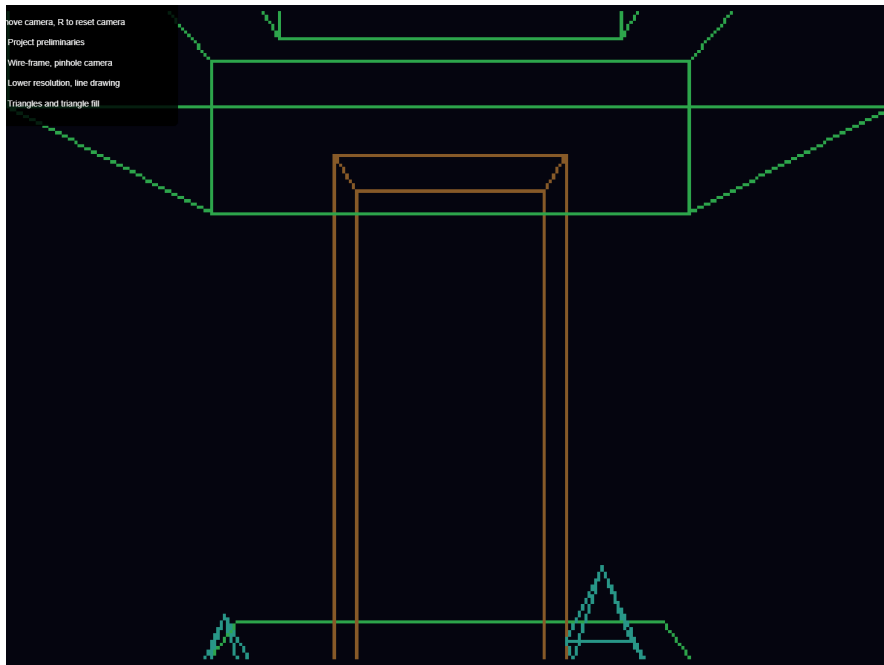
3 key - Level 3: Triangles and triangle fill

To complete Levels 0 and 1 (as well as scaffold levels 2 and 3), I added div tags to represent each of the different scenes (to satisfy each of the levels of the assignment) and listener functionality to allow key-based switching between the different methods of rendering/rasterization. The function “showScene” is called on each scene switch to set the current rasterization methods and show the required level.



At this stage, I also limited forward camera movement by adding a check. This only allowed the vector of the camera to be moved forward as long as it wasn't already 5 increments ahead of its starting position. By doing so, we

ensure that the camera doesn't move enough to flip the scene or cause unwanted interactions/lag. Left, Right, Backward, and reset via R were kept the same as the first assignment.



The pinhole camera-based vector calculation done in the first assignment was slightly modified to work with our scene builder method. Instead of looping through a global variable list of vertices and edges, it instead has a nested loop through the scene object instances, then accesses the vertices and edges for each.

Levels 2 and 3 required a custom line drawing method. Seeing as we had implemented Bresenham's algorithm in the in-class assignment, I chose to build off of that method. I kept the same implementation of the global pixel grid that is dynamically repainted on each button click. Using those functions, the draw function was updated to change which method it uses by checking the current scene ID.

Level 3 required the most adaptation, as the given example of rasterization uses a custom data type definition that seemed to not work with my JavaScript code. Once these errors were fixed, the implementation of the color filling was relatively simple. The design goal of the plan was for a pixel-style, solid-color aesthetic, so the code worked perfectly for this look.



3. Future Work

Admittedly, the goal of this project was to be a simple implementation. I have a tendency to be too ambitious with my project and not get enough done. That being said, there is definitely more that can be done to extend the scene and make it a full “walkable experience”. Taking the comforting and nostalgic feel of the scene, as well as the inspiration, into account, I would extend it to be a full picnic scene.

A first addition would have to be a dynamic light/shading engine to add shadows to the space. My idea to add this would be similar to the current implementation of the z-buffer for the camera, but it would be a separate buffer for the light source. Shadows could be calculated by multiplying the light source’s colors, the object’s colors, and the “shadow” color.

This multiplication would likely use dynamically applied weighting, similar to the pixel color weighting calculated in the barycentric rasterization guide provided. However, the math behind pixel-level calculation would be significantly more complex, as you would need to calculate light vectors, weighted by the position of the player and the angle of view of the camera.

Including more trees and bushes is obvious, but I would also want to add a motion engine. Specifically, one that cycles between a predefined set of pixel motions to add subtle animations to the bushes and leaves. Something like swaying in the wind. I would also use this pixel/line oscillation to animate a small campfire to place next to the picnic (I think in this art style it would look really nice). While I have no idea how, I suspect that this implementation would require an overhaul of the “scene-builder” function in my code.

These are the main changes I think would be nice for now. Anything more would require the full building of a game engine in javascript/html around the scene. While a fun project idea, I do feel that it would be very time-consuming.

key

